

Programming and Data Structures - II (CS3201)

Lecture 1

Kripabandhu Ghosh

CDS, IISER Kolkata

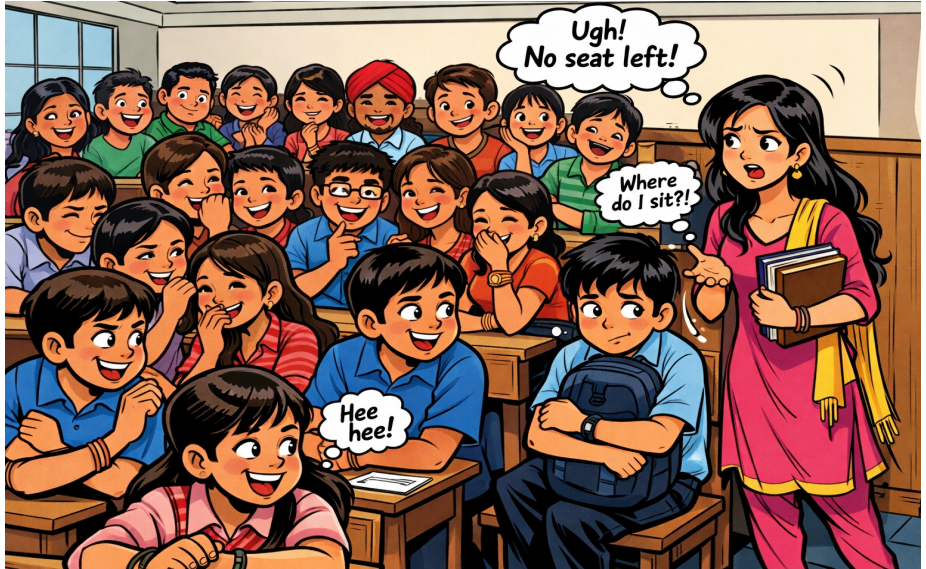
Story of life: Dynamism



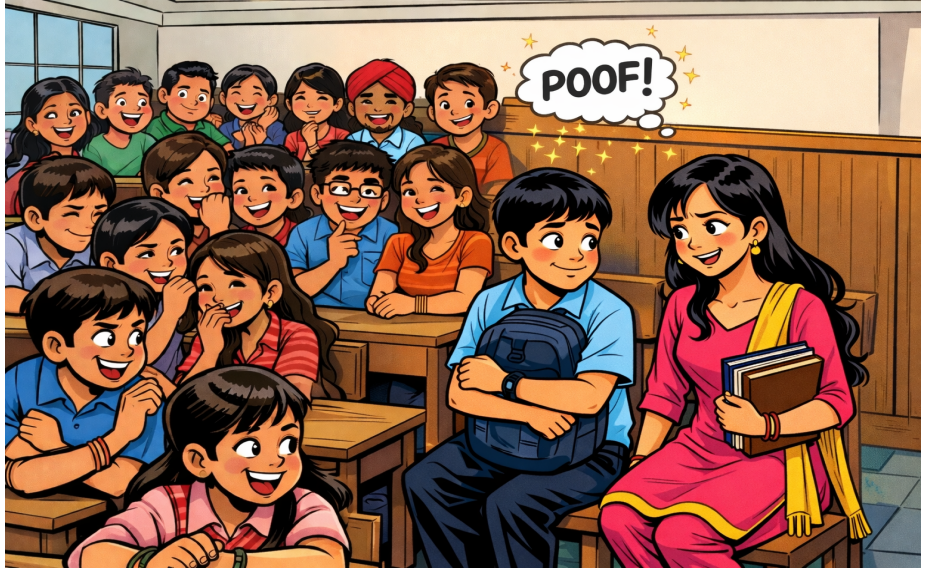
Story of workspace: Dynamism



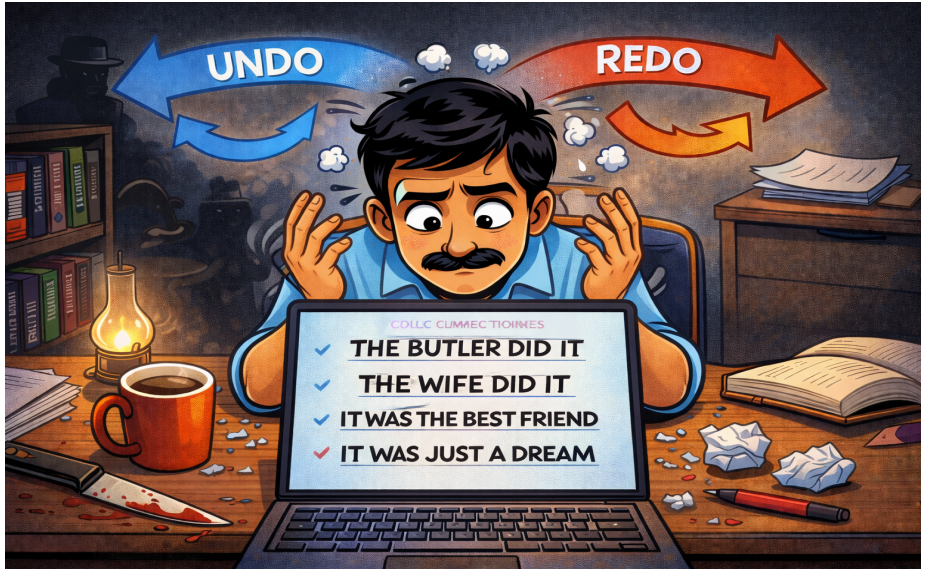
Story of static arrays: No seats left!



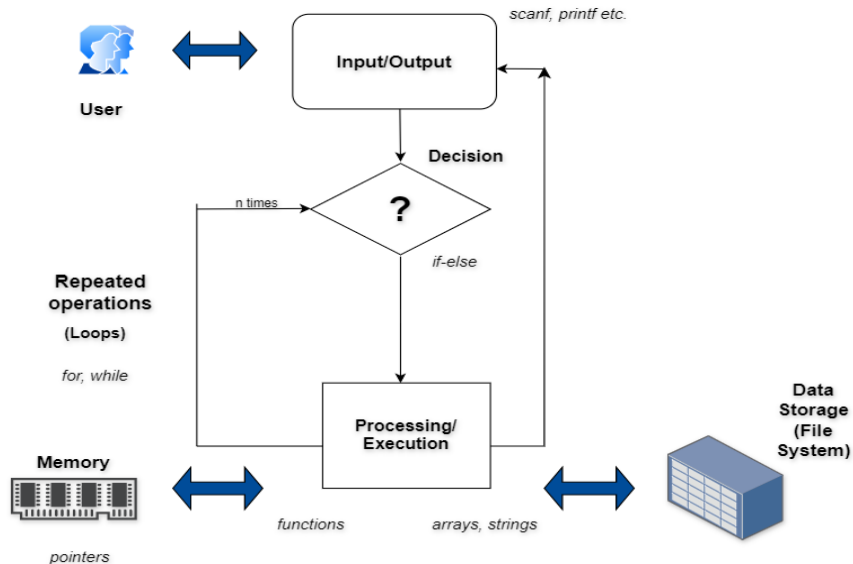
Story of dynamism: We have a new seat!



A thriller with multiple endings: UNDO-REDO operation



The Big Picture (Again!)



Introduction to the Instructors



Name: Saptarshi Pyne

Highest Qualification: PhD, Indian Institute of Technology, Kolkata, Guwahati (CSE)

Areas of Interest: Probabilistic and Causal Reasoning, Probabilistic Graphical Models, Bayesian Networks

Email: spyne [AT] iiserkol.ac.in

Introduction to the Instructors (contd.)



Name: Kripabandhu Ghosh

Highest Qualification: PhD, Indian Statistical Institute, Kolkata (CS)

Areas of Interest: Information Retrieval/Data Mining/AI on Legal Domain, Information Extraction from Social Media text, Fairness in Text Summarization, Machine Learning, Natural Language Processing

Email: kripaghosh [AT] iiserkol.ac.in

About the course CS3201

Advanced Data Structures and Algorithms

- Linked lists (Stack, Queue)
- Trees – Binary Search Tree, Heap, Priority Queue
- Hashing
- Dynamic Programming

About the course CS3201

Advanced Data Structures and Algorithms

- Linked lists (Stack, Queue)
- Trees – Binary Search Tree, Heap, Priority Queue
- Hashing
- Dynamic Programming

Implementation

C programming language

About the course CS3201

Advanced Data Structures and Algorithms

- Linked lists (Stack, Queue)
- Trees – Binary Search Tree, Heap, Priority Queue
- Hashing
- Dynamic Programming

Implementation

C programming language

Prerequisite

Decent knowledge of the basics of the C programming language

An advanced course...



Topic distribution

Refresher of the basic concepts of CS3101 Kripabandhu

Linked lists, Stack and queue using linked list Kripabandhu

Binary tree: traversal, search, Search Tree Saptarshi

Heap, Priority Queue, Huffman encoding Kripabandhu

Advanced trees, minimal spanning tree, shortest path Saptarshi

Hashing: functions, collision resolution Kripabandhu

Dynamic Programming Saptarshi

Total marks: 100

- Class tests
- End Sem

References

- Donald E. Knuth. *The Art of Computer Programming*, Vols 1 – 3.
- Cormen et al. *Introduction to Algorithms*, 3rd edition.
- Langsam, Augenstein, Tenenbaum. *Data Structures Using C and C++*.
- Samiran Chattopadhyay et al. *Data Structures Through C Language*

PREREQUISITE CONCEPTS

Prerequisite concepts

- Function (recursion)
- Structure
- Pointer
- Memory allocation (malloc)

USER-DEFINED FUNCTIONS

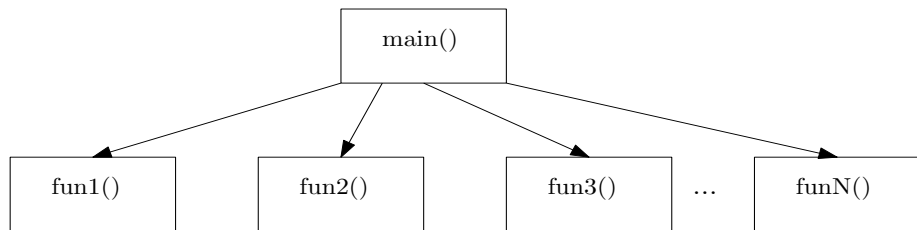
User-defined functions

Format

return-type function-name(argument-list)

- **return-type:** Data type returned by the function; default is integer ; can also be void (nothing returned)
 - **argument-list:** datatype arg1, datatype arg2,...
-
- A function may or may not have any arguments
 - A function may or may not return any value (maximum one value)

Functions: utility



Utilities

- Top-down modular programming
- **Decomposition** of a bigger problem into smaller ones, and **recomposition** from the smaller solutions
- Simplification of a complex logic
- Easier debugging

RECURSION

Recursion

- Function calls itself
- Solving a large (input size) problem by solving smaller related sub-problems

Conditions

- Base condition (terminating condition)
- Recursive/inductive condition

Recursive definition of factorial

Function elements

$$\mathit{factorial}(n) = \begin{cases} 1 & \text{if } n = 1 \\ n.\mathit{factorial}(n - 1) & \text{if } n > 1 \end{cases}$$

Factorial using recursion

$$\begin{aligned}\text{factorial}(n) &= n \times \text{factorial}(n - 1) \\ &= n \times (n - 1) \times \text{factorial}(n - 2) \\ &\dots \\ &= n \times (n - 1) \times \dots \times \text{factorial}(1) \\ &= n \times (n - 1) \times \dots \times 1\end{aligned}$$

$$n = 4$$

$$\begin{aligned}\text{factorial}(4) &= 4 \times \text{factorial}(3) \\ &= 4 \times 3 \times \text{factorial}(2) \\ &= 4 \times 3 \times 2 \times \text{factorial}(1) \\ &= 4 \times 3 \times 2 \times 1 \\ &= 24\end{aligned}$$

Factorial using recursion (program)

Program

```
#include <stdio.h>
```

```
int fact(int n) //Iterative version
```

```
{
    int i, f = 1;
    for(i = 1 ; i <= n; i++)
    {
        f = f*i;
    }
    return f;
}
```

```
int fact_recursive(n) //Recursive version
```

```
{
    int x;
    if(n == 1)
    {
        printf("Base condition return: n = %d\n", n);
        return 1;
    }
    else
    {
        printf("n = %d: before recursive call!\n", n);
        x = n*fact_recursive(n-1);
        printf("n = %d: after returning from call!\n", n);
        printf("Current value of factorial (x) = %d\n", x);
        return x;
    }
}
```

Factorial using recursion (program) (contd.)

Program

```
int main()
{
    int n = 4, fct, fct_rec;
    printf("Call from main\n");
    fct = fact(n);
    fct_rec = fact_recursive(n);
    printf("Return to main\n");
    printf("Factorial(%d) = %d (recursive)\n", n, fct_rec);
    printf("Factorial(%d) = %d (iterative)\n", n, fct);
    return 0;
}
```

Factorial using recursion (program) (contd.)

Output

Call from main

$n = 4$: before recursive call!

$n = 3$: before recursive call!

$n = 2$: before recursive call!

Base condition return: $n = 1$

$n = 2$: after returning from call!

Current value of factorial (x) = 2

$n = 3$: after returning from call!

Current value of factorial (x) = 6

$n = 4$: after returning from call!

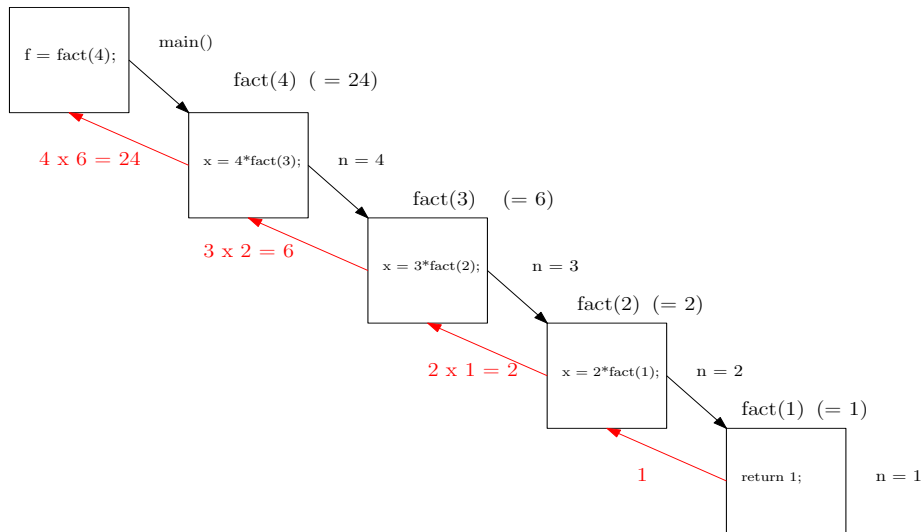
Current value of factorial (x) = 24

Return to main

Factorial(4) = 24 (recursive)

Factorial(4) = 24 (iterative)

Factorial using recursion (illustration)



Factorial: decomposition and recomposition

Base condition: $n = 1$

if $n == 1$, return value is 1

Inductive condition: $n > 1$

- Decomposition: $\text{fact_recursive}(n - 1)$
- Recomposition: $x = n * \text{fact_recursive}(n - 1)$

STRUCTURES

Structures

Format

```
struct tag_name
{
    data_type member1;
    data_type member2;
    ...
}
```

- collection of data items of different data types under the same name **tag_name**
- provision of having logically connected variables under the same name

Example

```
struct student
{
    char name[25];
    char roll_no[10];
    int semester_no;
};
```

Structures: declaration

```
struct student
{
    char name[25];
    char roll_no[10];
    int semester_no;
};
struct student student1, student2;
```

```
struct student
{
    char name[25];
    char roll_no[10];
    int semester_no;
} student1, student2;
```

```
struct
{
    char name[25];
    char roll_no[10];
    int semester_no;
} student1, student2;
```

Structures: member value access

Access

`struct_variable.member`

Example

`student1.name`
`student1.roll_no`

Structures: member value assignment

Assignment

```
strcpy(student1.name, "Kripa");  
strcpy(student1.roll_no, "F0172");  
student1.semester_no = 5;
```

Initialization

```
student2 = { "Kripa", "F0172", 5};
```

Copy

```
struct student student3;  
student3 = student1;
```

Structures: example

Program

```
#include<stdio.h>
#include<string.h>

struct student
{
    char name[25];
    char roll_no[10];
    int semester_no;
} student1;
```

Structures: example

Program

```
int main(void)
{
    struct student student2, student3 = {"Alok", "17MS345", 4}, student4;
    strcpy(student1.name, "Kripa");
    strcpy(student1.roll_no, "F0172");
    student1.semester_no = 5;
    student4 = student1; //Copy
    printf("Please input name, roll no and semester no below:\n");
    scanf("%s %s %d", student2.name, student2.roll_no, &student2.semester_no);
    printf("Student 1 record: %s %s %d\n", student1.name, student1.roll_no,
student1.semester_no);
    printf("Student 2 record: %s %s %d\n", student2.name, student2.roll_no,
student2.semester_no);
    printf("Student 3 record: %s %s %d\n", student3.name, student3.roll_no,
student3.semester_no);
    printf("Student 4 record: %s %s %d\n", student4.name, student4.roll_no,
student4.semester_no);
    printf("%lu %lu\n", sizeof(student1), sizeof(student2));
    return 0; }
```

Structures: example

Output

Please input name, roll no and semester no below:

Anuj

18MS123

5

Student 1 record: Kripa F0172 5

Student 2 record: Anuj 18MS123 5

Student 3 record: Alok 17MS345 4

Student 4 record: Kripa F0172 5

40 40

Structures: functions

Program

```
#include<stdio.h>
#include<string.h>

typedef struct book{
    char title[50];
    char id[10];
    int no;
}BOOK;

void display(BOOK b)
{
    printf("Book title: %s\n", b.title);
    printf("Book id: %s\n", b.id);
    printf("Copies in stock: %d\n\n", b.no);
}

BOOK update(BOOK b)
{
    b.no++;
    return b;
}
```


Structures: functions (contd.)

Program

```
int main(void)
{
    BOOK book1 = { "Let us C", "B123", 5}, book2 = { "The Art of Computer
Programming", "A1245", 3};
    display(book1);
    display(book2);
    book1 = update(book1);
    display(book1);
    return 0;
}
```

Structures: functions (contd.)

Output

Book title: Let us C

Book id: B123

Copies in stock: 5

Book title: The Art of Computer Programming

Book id: A1245

Copies in stock: 3

Book title: Let us C

Book id: B123

Copies in stock: 6

POINTERS

Variable

During the declaration of a variable (e.g. `int a = 5`), some location (address) is allocated in the memory to store the value of the variable

```
int a = 5;
```

a ← Variable

5 ← Value

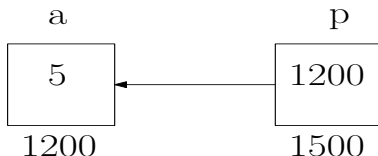
1200 ← Address

Pointers

Pointers are variables (e.g. `p`) which store the memory address of other variables (e.g. `a`)

- **Declaration:** `data_type *pointer_variable;` (e.g. `int *p;`)
 - **`data_type *`** indicates that `pointer_variable` is a pointer variable pointing to a variable of type `data_type`
- **Assignment:** `pointer_variable = &variable;` (e.g. `p = &a;`)
- **Address of:** `'&'` operator (`&variable` $\Rightarrow$ address of variable)

```
int a = 5;  
int *p;  
p = &a;
```



Pointer to structures: member value access

Access

`pointer_to_struct_variable->member`

Example

`BOOK* b`

`b->title`

`b->id`

Pointer to structures

Program

```
#include<stdio.h>
#include<string.h>

typedef struct book
{
    char title[50];
    char id[10];
    int no;
}BOOK;

void display(BOOK* b)
{
    printf("Book title: %s\n", b->title);
    printf("Book id: %s\n", b->id);
    printf("Copies in stock: %d\n", b->no);
}

void update(BOOK* b)
{
    b->no++;
}

int main(void)
{
    BOOK book1 = {"Let us C", "B123", 5}, book2 = {"The Art of Computer Programming", "A1245", 3};
    display(&book1);
    display(&book2);
    update(&book1);
    display(&book1);
    return 0;
}
```

Pointer to structures (contd.)

Output

Book title: Let us C

Book id: B123

Copies in stock: 5

Book title: The Art of Computer Programming

Book id: A1245

Copies in stock: 3

Book title: Let us C

Book id: B123

Copies in stock: 6

Pointers and Arrays: Memory allocation

Contiguous memory allocation

`int a[5];` `a[0]` `a[1]` `a[2]` `a[3]` `a[4]` ← Elements

5	6	1	2	9
---	---	---	---	---

← Values

1000 1004 1008 1012 1016 ← Addresses

Pointers and Arrays: Access

Premise

```
int a[] = {5, 6, 1, 2, 9};  
int *p;  
p = a;
```

Pointer expression	Interpretation	Remarks
<code>p = a;</code>	<code>p = &a[0]</code>	Pointing to the starting address of the first element of array a
<code>(p + i)</code>	<code>&a[i]</code>	Accessing array element address via pointer
<code>*(p + i)</code>	<code>a[i]</code>	Accessing array element value via pointer
<code>p++</code>	<code>a[++i]</code>	Pointer shifts to the next array element (next element of the same data type)
<code>&a[i]</code>	base address + (i * sizeof(int))	base address: <code>&a[0]</code>
<code>*(a + i)</code>	<code>a[i]</code>	array as a pointer

Pointers and Arrays: Access

Program

```
#include<stdio.h>
#include<string.h>

int main()
{
    int a[] = {5, 6, 1, 2, 9}, *p, i;
    printf("Sizeof int = %zu\n", sizeof(int));
    printf("Accessing using array indices:\n\n");
    for(i = 0; i < 5; i++)
    {
        printf("Value = %d at address = %u\n", a[i], &a[i]);
    }
    p = &a[0]; //p = a also works
    i = 0;
    printf("Accessing using pointers:\n\n");
    while(i < 5)
    {
        printf("Array: Value = %d at address = %u\n", a[i], &a[i]);
        printf("Pointer *(p + i): Value = %d at address = %u\n", *(p + i), p);
        printf("Pointer *(i + p): Value = %d at address = %u\n", *(i + p), p);
        printf("Pointer *(a + i): Value = %d at address = %u\n", *(a + i), (a + i));
        printf("Pointer *(i + a): Value = %d at address = %u\n", *(i + a), (i + a));
        printf("Pointer i[a]: Value = %d at address = %u\n", i[a], &i[a]);
        printf("\n===== \n");
        i++;
    }
    return 0;
}
```

Pointers and Arrays: Access

Output

Sizeof int = 4

Accessing using array indices:

Value = 5 at address = 1513737520

Value = 6 at address = 1513737524

Value = 1 at address = 1513737528

Value = 2 at address = 1513737532

Value = 9 at address = 1513737536

Pointers and Arrays: Access

Output (contd.)

Accessing using pointers:

Array: Value = 5 at address = 1513737520

Pointer $*(p + i)$: Value = 5 at address = 1513737520

Pointer $*(i + p)$: Value = 5 at address = 1513737520

Pointer $*(a + i)$: Value = 5 at address = 1513737520

Pointer $*(i + a)$: Value = 5 at address = 1513737520

Pointer $i[a]$: Value = 5 at address = 1513737520

=====

Pointers and Arrays: Access

Output (contd.)

Array: Value = 6 at address = 1513737524

Pointer $*(p + i)$: Value = 6 at address = 1513737524

Pointer $*(i + p)$: Value = 6 at address = 1513737524

Pointer $*(a + i)$: Value = 6 at address = 1513737524

Pointer $*(i + a)$: Value = 6 at address = 1513737524

Pointer $i[a]$: Value = 6 at address = 1513737524

=====

Array: Value = 1 at address = 1513737528

Pointer $*(p + i)$: Value = 1 at address = 1513737528

Pointer $*(i + p)$: Value = 1 at address = 1513737528

Pointer $*(a + i)$: Value = 1 at address = 1513737528

Pointer $*(i + a)$: Value = 1 at address = 1513737528

Pointer $i[a]$: Value = 1 at address = 1513737528

=====

Pointers and Arrays: Access

Output (contd.)

Array: Value = 2 at address = 1513737532

Pointer $*(p + i)$: Value = 2 at address = 1513737532

Pointer $*(i + p)$: Value = 2 at address = 1513737532

Pointer $*(a + i)$: Value = 2 at address = 1513737532

Pointer $*(i + a)$: Value = 2 at address = 1513737532

Pointer $i[a]$: Value = 2 at address = 1513737532

=====

Array: Value = 9 at address = 1513737536

Pointer $*(p + i)$: Value = 9 at address = 1513737536

Pointer $*(i + p)$: Value = 9 at address = 1513737536

Pointer $*(a + i)$: Value = 9 at address = 1513737536

Pointer $*(i + a)$: Value = 9 at address = 1513737536

Pointer $i[a]$: Value = 9 at address = 1513737536

=====

Pointers and Arrays: Access (shift)

Program

```
#include<stdio.h>
#include<string.h>

int main()
{
    int a[] = {5, 6, 1, 2, 9}, *p, i;
    p = &a[0]; //p = a also works
    i = 0;
    printf("Accessing using pointers:\n\n");
    while(i < 5)
    {
        printf("Pointer *p: Value = %d at address = %u\n", *p, p);
        p++;
        i++;
    }
    return 0;
}
```


Pointers and Arrays: Access (shift)

Output

Accessing using pointers:

Pointer *p: Value = 5 address = 2520572032

Pointer *p: Value = 6 address = 2520572036

Pointer *p: Value = 1 address = 2520572040

Pointer *p: Value = 2 address = 2520572044

Pointer *p: Value = 9 address = 2520572048

Pointers and Strings in function calls

Program

```
#include<stdio.h>
#include<string.h>
#include<stdlib.h>

char* reverseString(char* string) //Function that returns the reversed version of string
{
    char *reversed;
    int i = 0, len;
    len = strlen(string);
    reversed = (char*)malloc((len + 1)*sizeof(char)); //Allocated memory for the reversed string
    while(string[i] != '\0')
    {
        reversed[len - 1 - i] = string[i];
        i++;
    }
    reversed[i] = '\0';
    return reversed;
}

int main()
{
    char* name = "Kripa";
    printf("Reversed version of %s is %s\n", name, reverseString(name));
    return 0;
}
```

Output

Reversed version of Kripa is apirK

Pointers and Strings in function calls: const

Program

```
#include<stdio.h>
#include<string.h>
#include<stdlib.h>

void xstrcpy(char* copy, const char* string) //Function that copies 'string' to 'copy'
{
    while(*string != '\0')
    {
        *copy = *string;
        copy++;
        string++;
    }
    *copy = "\0";
    //string[0] = 'I'; //Not allowed
}

int main()
{
    char* name = "Kripa", copy[20];
    xstrcpy(copy, name);
    printf("The copy of %s is %s\n", name, copy);
    return 0;
}
```

Output

The copy of Kripa is Kripa

Note

The *const* (constant) qualifier prohibits changes in the value of the associated variable

Two-dimensional Arrays

Format: declaration

```
type variable_name[no_of_rows][no_of_cols]
```

- **type**: data type of each element of the array (e.g. int, float etc.)
- **no_of_rows**: number of row elements in the array
- **no_of_cols**: number of columns elements in the array

Example

```
int marks[10][5];
```

Two-dimensional Arrays

Format: access

marks[*i*][*j*]: the element at i^{th} row and j^{th} column of the array *marks* ($0 \leq i < \text{no_of_rows}$, $0 \leq j < \text{no_of_cols}$)

marks[0][0] *marks*[0][1] ... *marks*[0][*j*] ... *marks*[0][no_of_cols - 1]

marks[1][0] *marks*[1][1] ... *marks*[1][*j*] ... *marks*[1][no_of_cols - 1]

...

marks[*i*][0] *marks*[*i*][1] ... *marks*[*i*][*j*] ... *marks*[*i*][no_of_cols - 1]

...

marks[no_of_rows - 1][0] *marks*[no_of_rows - 1][1] ... *marks*[no_of_rows - 1][*j*] ...

marks[no_of_rows - 1][no_of_cols - 1]

Two-dimensional Arrays: Initialization

```
A[3][2] = {{2, 5}, {3, 1}, {7, 4}};
```

```
A[3][2] = {2, 5, 3, 1, 7, 4};
```

```
A[3][2] = {{2, 5},  
           {3, 1},  
           {7, 4}};
```

```
A[3][2] = {};
```

Matrix Multiplication

$$\begin{array}{ccc} \text{A} & & \text{B} \\ \begin{array}{|c|c|c|} \hline 2 & 1 & 2 \\ \hline 3 & 1 & 3 \\ \hline 2 & 5 & 1 \\ \hline \end{array} & \times & \begin{array}{|c|c|} \hline 2 & 5 \\ \hline 3 & 1 \\ \hline 7 & 4 \\ \hline \end{array} \\ \begin{array}{c} \overrightarrow{\hspace{1.5cm}} \\ 3 \times 3 \end{array} & & \begin{array}{c} \overrightarrow{\hspace{1.5cm}} \\ 3 \times 2 \end{array} \end{array} = \begin{array}{|c|c|} \hline 21 & 19 \\ \hline 30 & 28 \\ \hline 26 & 19 \\ \hline \end{array} \begin{array}{c} \text{Prod} \\ 3 \times 2 \end{array}$$

Matrix Multiplication: program

Program

```
#include<stdio.h>

void main()
{
    int A[3][3] = { {2, 1, 2}, {3, 1, 3}, {2, 5, 1} }, B[3][2] = { {2, 5}, {3, 1}, {7, 4} }, Prod[10][10] = { }, nr1 = 3, nc1 = 3, nr2 = 3, nc2 = 2, nr3,
    nc3, i, j, k;
    if(nc1 != nr2)
    {
        printf("Invalid multiplication!\n");
    }
    else
    {
        for(i = 0 ; i < nr1 ; i++)
        {
            for(j = 0 ; j < nc2 ; j++)
            {
                for(k = 0 ; k < nc1 ; k++)
                {
                    Prod[i][j] += A[i][k]*B[k][j];
                }
            }
        }
        printf("The product:\n");
        nr3 = nr1;
        nc3 = nc2;
        for(i = 0 ; i < nr1 ; i++)
        {
            for(j = 0 ; j < nc2 ; j++)
            {
                printf("%d ", Prod[i][j]);
            }
            printf("\n");
        }
    }
}
```


Matrix Multiplication: program (contd.)

Output

The product:

21 19

30 28

26 19

Multi-dimensional Arrays

Format

type variable_name[s1][s2][s3]...[sn]

- **type**: data type of each element of the array (e.g. int, float etc.)
- **si**: size of the i^{th} dimension

Example

```
int marks[10][5][20];  
float chart[20][2][30][25];
```

Graphs and Matrices¹

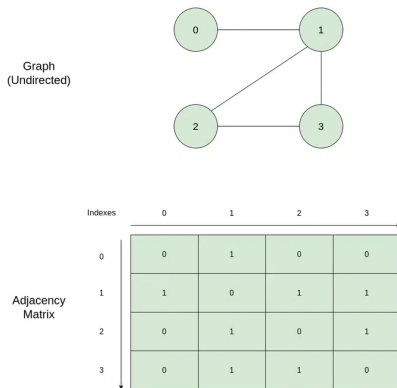
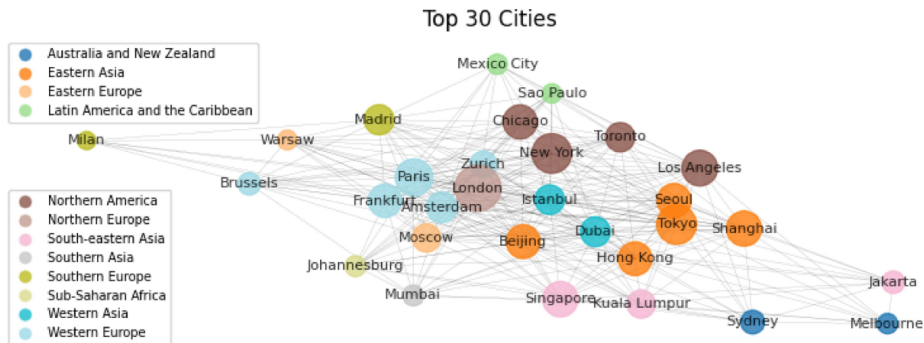


Figure: Matrix Representation of an Undirected Graph

¹<https://www.geeksforgeeks.org/java/graph-adjacency-matrix-in-java/>

Graphs and Cities²

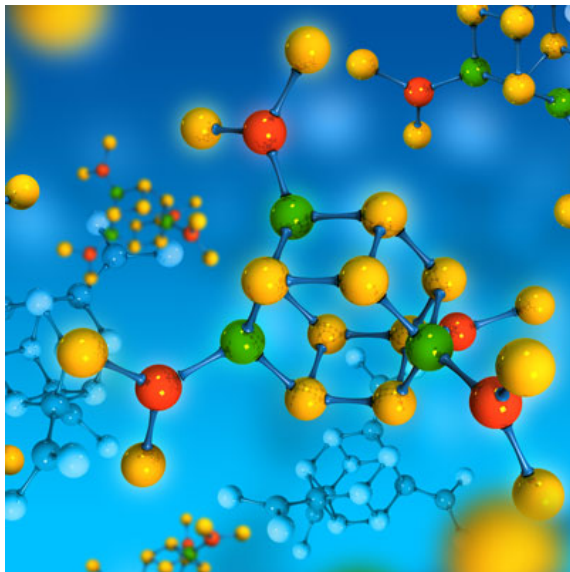


²<https://www.nature.com/articles/s41598-024-69494-1>

Graphs and Google Map³



Graphs and Molecules⁴



⁴<https://mathigon.org/course/graph-theory/applications>

Pointers and Matrices

Premise

```
int a[2][3] = {{5, 6, 1}, {2, 9, 7}}, *p, (*q)[3], *r;  
p = (int*)a;  
q = a;  
r = q + i;
```

Pointer expression	Interpretation
$a + i$	Pointer to the i th row
$*(a + i)$	Pointer to the first element of the i th row
$*(a + i) + j$	Pointer to the j th element of the i th row
$*(*(a + i) + j)$	$a[i][j]$
$*(p + i * \text{noOfCols} + j)$	$a[i][j]$
$*(r + j)$	$a[i][j]$
$(*q)[3]$	Pointer to an array of 3 integers

Pointers and Matrices: implementation

Program

```
#include<stdio.h>
#include<string.h>
int main()
{
    int a[2][3] = { {5, 6, 1}, {2, 9, 7} }, *p, (*q)[3], *r, i, j, noOfRows = 2, noOfCols = 3;
    printf("Printing the element-wise addresses:\n");
    for(i = 0; i < noOfRows; i++)
    {
        for(j = 0; j < noOfCols; j++)
        {
            printf("%u ", &a[i][j]);
        }
        printf("\n");
    }
    printf("\n\n");
    printf("Pointers to 0th row, 1st row: %u %u\n", &a[0][0], &a[1][0]);
    printf("Pointers to 0th row, 1st row: %u %u\n", *a, *(a + 1));
    printf("Pointers to the (0,1)th, (1,1)th elements: %u %u\n", &a[0][1], &a[1][1]);
    printf("Pointers to the (0,1)th, (1,1)th elements: %u %u\n", *(a + 0) + 1, *(a + 1) + 1);
    printf("\n\n");
    p = (int*)a;
    printf("Using a in pointer form...\n\n");
    for(i = 0; i < noOfRows; i++)
    {
        for(j = 0; j < noOfCols; j++)
        {
            printf("%d ", (*(a + i) + j));
        }
        printf("\n");
    }
}
```


Pointers and Matrices: implementation (contd.)

Program (contd.)

```
printf("Using p...\n\n");
for(i = 0; i < noOfRows; i++)
{
    for(j = 0; j < noOfCols; j++)
    {
        printf("%d ", *(p + i*noOfCols + j));
    }
    printf("\n");
}
return 0;
}
```

Pointers and Matrices: implementation (contd.)

Output

Printing the element-wise addresses:

1843915904 1843915908 1843915912

1843915916 1843915920 1843915924

Pointers to 0th row, 1st row: 1843915904 1843915916

Pointers to 0th row, 1st row: 1843915904 1843915916

Pointers to the (0,1)th, (1,1)th elements: 1843915908 1843915920

Pointers to the (0,1)th, (1,1)th elements: 1843915908 1843915920

Pointers and Matrices: visualization

